64-Stage AI Capability Roadmap

A stage-by-stage build and approval checklist for a multimodal AI system

Each stage is meant to be built, tested, reviewed, and approved before the next stage begins.

How to use this document: Treat every stage as a gated milestone. Build only the in-scope capability, run the stage-specific tests, review failures, and do not advance until the stage is approved.

Important note: This roadmap is a practical engineering breakdown of AGI-like capability areas, not proof that a final system is AGI. Sequence may be adjusted, but stage gating should stay strict.

Prepared for: staged local AI development
Date: April 11, 2026

Roadmap Rules

- **[ ]** One active stage at a time. Unfinished capabilities must stay disabled or absent.
- **[ ]** Every stage needs a written definition, a reproducible test set, and an approval decision.
- **[ ]** Regression testing is mandatory. A new stage does not count as success if it breaks an earlier approved stage.
- **[ ]** Keep logs, failure examples, and reset procedures from the beginning. Hidden state makes approval weak.
- **[ ]** Prefer narrow, inspectable implementations over clever but opaque ones.

Capability Bands

Stages	Band	Purpose	Typical Gate
1-12	Foundational cognition	Breadth, transfer, reasoning, planning, decomposition, working context	Can the core cognitive behavior be demonstrated on novel tasks?
13-23	Memory, learning, and self-correction	Remember, retrieve, predict, reflect, learn, revise	Can state be trusted, inspected, reset, and improved safely?
24-30	Multimodal communication and interaction	Understand, generate, converse, use tools, take bounded action	Does the same intent remain coherent across modalities and tools?
31-38	Agency, planning, and execution	Autonomy, instruction discipline, general problem solving	Can the system act more independently without losing control?
39-48	Social alignment, safety, and epistemics	Social fit, alignment, ethics, truthfulness, confidence	Can users trust the system's behavior under pressure?
49-64	Robustness, systems maturity, and scale	Stress tolerance, consistency, explainability, auditability, integration	Can the capability survive noise, long runs, and model changes?

64-Stage Index

Stage	Ability	Band
1	Broad Generality	Foundational cognition
2	Transfer Learning	Foundational cognition
3	Rapid Adaptation	Foundational cognition
4	Few-Shot or Zero-Shot Competence	Foundational cognition

5	Robust Reasoning	Foundational cognition
6	Common-Sense Understanding	Foundational cognition
7	Abstract Thinking	Foundational cognition
8	Causal Reasoning	Foundational cognition
9	Long-Horizon Planning	Foundational cognition
10	Goal-Directed Behavior	Foundational cognition
11	Subgoal Decomposition	Foundational cognition
12	Working Memory	Foundational cognition
13	Long-Term Memory	Memory, learning, and self-correction
14	Retrieval and Use of Prior Knowledge	Memory, learning, and self-correction
15	World Modeling	Memory, learning, and self-correction
16	Prediction	Memory, learning, and self-correction
17	Uncertainty Awareness	Memory, learning, and self-correction
18	Metacognition	Memory, learning, and self-correction
19	Error Correction	Memory, learning, and self-correction
20	Self-Monitoring	Memory, learning, and self-correction
21	Learning from Experience	Memory, learning, and self-correction
22	Continual Learning	Memory, learning, and self-correction
23	Sample Efficiency	Memory, learning, and self-correction
24	Multimodal Understanding	Multimodal communication and interaction
25	Multimodal Generation	Multimodal communication and interaction
26	Language Competence	Multimodal communication and interaction
27	Pragmatic Communication	Multimodal communication and interaction
28	Interactive Dialogue	Multimodal communication and interaction
29	Tool Use	Multimodal communication and interaction

30	Environment Interaction	Multimodal communication and interaction
31	Autonomy	Agency, planning, and execution
32	Instruction Following	Agency, planning, and execution
33	Constraint Satisfaction	Agency, planning, and execution
34	General Problem Solving	Agency, planning, and execution
35	Creativity	Agency, planning, and execution
36	Scientific Reasoning	Agency, planning, and execution
37	Mathematical Competence	Agency, planning, and execution
38	Strategic Reasoning	Agency, planning, and execution
39	Social Intelligence	Social alignment, safety, and epistemics
40	Theory of Mind	Social alignment, safety, and epistemics
41	Personalization	Social alignment, safety, and epistemics
42	Value Alignment	Social alignment, safety, and epistemics
43	Safety Awareness	Social alignment, safety, and epistemics
44	Ethical Sensitivity	Social alignment, safety, and epistemics
45	Truth-Seeking Behavior	Social alignment, safety, and epistemics
46	Hallucination Resistance	Social alignment, safety, and epistemics
47	Calibration	Social alignment, safety, and epistemics
48	Reliability	Social alignment, safety, and epistemics
49	Robustness	Robustness, systems maturity, and scale
50	Generalization to Novelty	Robustness, systems maturity, and scale

51	Compositionality	Robustness, systems maturity, and scale
52	Temporal Coherence	Robustness, systems maturity, and scale
53	Identity and Role Stability	Robustness, systems maturity, and scale
54	Self-Contained Task Management	Robustness, systems maturity, and scale
55	Explainability	Robustness, systems maturity, and scale
56	Auditability	Robustness, systems maturity, and scale
57	Preference Learning	Robustness, systems maturity, and scale
58	Negotiation of Ambiguity	Robustness, systems maturity, and scale
59	Embodied Competence	Robustness, systems maturity, and scale
60	Resource Awareness	Robustness, systems maturity, and scale
61	Self-Improvement Ability	Robustness, systems maturity, and scale
62	Cross-Domain Integration	Robustness, systems maturity, and scale
63	Human-Comparable Task Range	Robustness, systems maturity, and scale
64	Minimal Dependence on Task-Specific Engineering	Robustness, systems maturity, and scale

Foundational cognition

These stages establish the baseline cognitive engine: breadth, transfer, reasoning, planning, decomposition, and short-context state handling.

Gate discipline: Treat this as a capability-isolation stage. The objective is to prove the core cognitive behavior exists, not to build a polished product experience around it.

Stage 1 — Broad Generality

Stage intent	Support a wide spread of task types without bespoke task-specific pipelines.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making broad generality explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for broad generality with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Broad Generality' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate broad generality from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates broad generality on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.

- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for broad generality.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 2 — Transfer Learning

Stage intent	Reuse knowledge learned in one domain when solving problems in another.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making transfer learning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for transfer learning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Transfer Learning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.

- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate transfer learning from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates transfer learning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for transfer learning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 3 — Rapid Adaptation

Stage intent	Adjust quickly to new tasks, rules, and environments with small amounts of feedback.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making rapid adaptation explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for rapid adaptation with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Rapid Adaptation' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate rapid adaptation from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates rapid adaptation on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for rapid adaptation.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 4 — Few-Shot or Zero-Shot Competence

Stage intent	Perform unfamiliar tasks from instructions alone or from a handful of demonstrations.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making few-shot or zero-shot competence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for few-shot or zero-shot competence with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Few-Shot or Zero-Shot Competence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate few-shot or zero-shot competence from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates few-shot or zero-shot competence on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.

- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for few-shot or zero-shot competence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 5 — Robust Reasoning

Stage intent	Solve novel problems through explicit reasoning rather than pattern mimicry alone.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making robust reasoning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for robust reasoning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Robust Reasoning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate robust reasoning from generic eloquence or prompt tricks.

- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates robust reasoning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for robust reasoning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 6 — Common-Sense Understanding

Stage intent	Model everyday facts about objects, people, physics, and social situations.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making common-sense understanding explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for common-sense understanding with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.

- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Common-Sense Understanding' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate common-sense understanding from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates common-sense understanding on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for common-sense understanding.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 7 — Abstract Thinking

Stage intent	Manipulate concepts, analogies, hierarchies, and symbolic structure.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making abstract thinking explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for abstract thinking with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Abstract Thinking' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate abstract thinking from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates abstract thinking on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for abstract thinking.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 8 — Causal Reasoning

Stage intent	Separate cause from correlation and reason about interventions and counterfactuals.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making causal reasoning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for causal reasoning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Causal Reasoning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate causal reasoning from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.

- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates causal reasoning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for causal reasoning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 9 — Long-Horizon Planning

Stage intent	Build and maintain plans that span many steps and delayed rewards.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making long-horizon planning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for long-horizon planning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.

- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Long-Horizon Planning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate long-horizon planning from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates long-horizon planning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for long-horizon planning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 10 — Goal-Directed Behavior

Stage intent	Select actions that continue to optimize toward a defined objective.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making

	goal-directed behavior explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for goal-directed behavior with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Goal-Directed Behavior' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate goal-directed behavior from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates goal-directed behavior on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for goal-directed behavior.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 11 — Subgoal Decomposition

Stage intent	Break large tasks into smaller ordered pieces with clear dependencies.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making subgoal decomposition explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for subgoal decomposition with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Subgoal Decomposition' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate subgoal decomposition from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates subgoal decomposition on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for subgoal decomposition.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 12 — Working Memory

Stage intent	Keep relevant intermediate facts active while reasoning and acting.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making working memory explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for working memory with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A prompt set covering easy, medium, and adversarial reasoning examples.
- **[ ]** A simple scorecard showing pass rate, failure categories, and regression status.

Implementation checklist

- **[ ]** Define the operational meaning of 'Working Memory' in one page or less so the stage is testable instead of vague.

- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Create task sets that isolate working memory from generic eloquence or prompt tricks.
- **[ ]** Separate capability from memorized benchmark contamination by emphasizing novel and remixed test cases.
- **[ ]** Tune for correctness first, then speed and convenience.

Test checklist

- **[ ]** Verify the system demonstrates working memory on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include at least a few out-of-distribution examples that cannot be solved by surface pattern matching alone.
- **[ ]** Inspect reasoning traces or intermediate artifacts when available, but score the final behavior rather than rhetoric.

Approval gate

- **[ ]** You can explain in plain language what counts as success for working memory.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Performance reflects real competence rather than lucky prompting or memorized examples.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Memory, learning, and self-correction

These stages move from one-shot responses toward a system that can remember, retrieve, predict, reflect, learn, and revise.

Gate discipline: Every write path needs a reset path. Hidden, irreversible state changes make debugging and approval difficult.

Stage 13 — Long-Term Memory

Stage intent	Store and retrieve durable knowledge, preferences, and prior interactions over time.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making long-term memory explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for long-term memory with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Long-Term Memory' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates long-term memory on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for long-term memory.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 14 — Retrieval and Use of Prior Knowledge

Stage intent	Recall the right stored information at the right moment and apply it correctly.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making retrieval and use of prior knowledge explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for retrieval and use of prior knowledge with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Retrieval and Use of Prior Knowledge' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates retrieval and use of prior knowledge on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for retrieval and use of prior knowledge.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 15 — World Modeling

Stage intent	Maintain an internal model of how the environment, agents, and systems behave.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making world modeling explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or

	reject.

Minimum deliverables

- **[ ]** Stage specification for world modeling with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'World Modeling' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates world modeling on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for world modeling.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 16 — Prediction

Stage intent	Forecast likely future states, outcomes, and downstream effects.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making prediction explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for prediction with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Prediction' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates prediction on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for prediction.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 17 — Uncertainty Awareness

Stage intent	Detect and express uncertainty instead of acting falsely certain.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making uncertainty awareness explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for uncertainty awareness with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Uncertainty Awareness' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.

- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates uncertainty awareness on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for uncertainty awareness.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 18 — Metacognition

Stage intent	Inspect one's own reasoning process and notice likely failure modes.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making metacognition explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for metacognition with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Metacognition' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates metacognition on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for metacognition.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 19 — Error Correction

Stage intent	Recover from mistakes through detection, revision, and retry.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making error correction explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for error correction with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Error Correction' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates error correction on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for error correction.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 20 — Self-Monitoring

Stage intent	Track whether current behavior still matches the task, policy, and context.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making self-monitoring explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for self-monitoring with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Self-Monitoring' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates self-monitoring on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for self-monitoring.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.

- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 21 — Learning from Experience

Stage intent	Improve from successes, failures, and explicit feedback after deployment.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making learning from experience explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for learning from experience with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Learning from Experience' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates learning from experience on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for learning from experience.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 22 — Continual Learning

Stage intent	Add new capabilities without catastrophic forgetting of prior ones.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making continual learning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for continual learning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Continual Learning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates continual learning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for continual learning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 23 — Sample Efficiency

Stage intent	Improve meaningfully from limited data instead of requiring huge retraining sets.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making sample efficiency explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for sample efficiency with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A memory or learning-state schema with reset and inspection controls.
- **[ ]** A regression suite proving that added memory or learning does not silently corrupt earlier stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Sample Efficiency' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Specify when the system should write, update, ignore, or delete internal state.
- **[ ]** Guard against stale, duplicated, or contradictory memory entries.
- **[ ]** Make rollback easy so a bad learning step can be reversed without retraining the whole system.

Test checklist

- **[ ]** Verify the system demonstrates sample efficiency on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test reset, rollback, and state-isolation behavior explicitly.
- **[ ]** Probe contradiction cases where old memory should be corrected or deprioritized.

Approval gate

- **[ ]** You can explain in plain language what counts as success for sample efficiency.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** State behavior is inspectable, reversible, and does not silently poison later outputs.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Multimodal communication and interaction

These stages extend the system beyond plain text into multi-modal understanding, generation, dialogue, tools, and environmental action.

Gate discipline: Do not add a new modality or tool path unless you can show the model grounds it correctly and fails safely when the signal is weak.

Stage 24 — Multimodal Understanding

Stage intent	Interpret information jointly across text, images, audio, video, and structured inputs.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making multimodal understanding explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for multimodal understanding with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Multimodal Understanding' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates multimodal understanding on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for multimodal understanding.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 25 — Multimodal Generation

Stage intent	Produce useful outputs in multiple modalities with consistent meaning.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making multimodal generation explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for multimodal generation with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Multimodal Generation' in one page or less so the stage is testable instead of vague.

- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates multimodal generation on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for multimodal generation.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 26 — Language Competence

Stage intent	Understand and generate language with precision, fluency, and context awareness.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making language competence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for language competence with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Language Competence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates language competence on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for language competence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 27 — Pragmatic Communication

Stage intent	Infer intent, implication, audience needs, and conversational subtext.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making pragmatic communication explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for pragmatic communication with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Pragmatic Communication' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates pragmatic communication on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for pragmatic communication.

- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 28 — Interactive Dialogue

Stage intent	Sustain coherent, context-aware multi-turn conversations.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making interactive dialogue explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for interactive dialogue with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Interactive Dialogue' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates interactive dialogue on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for interactive dialogue.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 29 — Tool Use

Stage intent	Select and use external tools, APIs, and software when they improve task performance.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making tool use explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for tool use with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Tool Use' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates tool use on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for tool use.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 30 — Environment Interaction

Stage intent	Take actions in digital or physical environments and observe their effects.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making environment interaction explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure

	log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for environment interaction with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A clear input/output contract for each supported modality or tool path.
- **[ ]** A fallback path for unsupported inputs so the system degrades gracefully.

Implementation checklist

- **[ ]** Define the operational meaning of 'Environment Interaction' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Normalize inputs from each modality into a shared internal representation where possible.
- **[ ]** Preserve provenance so the system knows what came from text, image, audio, tool output, or user instruction.
- **[ ]** Handle unsupported media, missing fields, and partial tool failures without crashing.

Test checklist

- **[ ]** Verify the system demonstrates environment interaction on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test modality mismatch and cross-modal grounding, not just single-modality success.
- **[ ]** Measure whether the same intent is preserved when the input format changes.

Approval gate

- **[ ]** You can explain in plain language what counts as success for environment interaction.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** Cross-modal or tool-linked behavior remains coherent when the same task is presented in multiple forms.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.

- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Agency, planning, and execution

These stages push the assistant from reactive chat into bounded autonomy, disciplined execution, and general problem solving.

Gate discipline: Bounded autonomy is the rule. Every action path should remain observable, interruptible, and attributable.

Stage 31 — Autonomy

Stage intent	Operate for extended stretches with limited hand-holding while staying bounded.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making autonomy explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for autonomy with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'Autonomy' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates autonomy on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.

- [ ] Repeat key tests across at least three separate runs to check stability.
- [ ] Log every failure with the input, observed output, expected behavior, and probable cause.
- [ ] Confirm no previously approved stage regressed materially after this capability was added.
- [ ] Test interrupted runs, invalid tools, and missing permissions.
- [ ] Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- [ ] You can explain in plain language what counts as success for autonomy.
- [ ] The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- [ ] Results are good enough that you would trust the feature in its current bounded scope.
- [ ] The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- [ ] Passes only on cherry-picked prompts or a single benchmark family.
- [ ] Failure cases are not cataloged well enough to reproduce and fix.
- [ ] You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- [ ] The new stage materially worsens any previously approved stage.

Stage 32 — Instruction Following

Stage intent	Follow detailed user instructions, constraints, and formatting requirements reliably.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making instruction following explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- [ ] Stage specification for instruction following with a strict in-scope / out-of-scope boundary.
- [ ] A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- [ ] A reproducible test script or benchmark harness dedicated to this stage.
- [ ] A failure log capturing at least the top recurring breakdowns observed during testing.
- [ ] An execution boundary defining what the system may do automatically and what still requires user confirmation.
- [ ] Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- [ ] Define the operational meaning of 'Instruction Following' in one page or less so the stage is testable instead of vague.
- [ ] Set one minimum success metric and one stretch metric before implementation begins.
- [ ] Implement the smallest version that can pass a controlled test without depending on later stages.

- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates instruction following on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for instruction following.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 33 — Constraint Satisfaction

Stage intent	Respect budgets, rules, safety limits, dependencies, and hard requirements.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making constraint satisfaction explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for constraint satisfaction with a strict in-scope / out-of-scope boundary.

- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'Constraint Satisfaction' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates constraint satisfaction on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for constraint satisfaction.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 34 — General Problem Solving

Stage intent	Solve unfamiliar problems across domains using

	reusable cognitive machinery.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making general problem solving explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for general problem solving with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'General Problem Solving' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates general problem solving on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for general problem solving.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 35 — Creativity

Stage intent	Generate useful, original ideas rather than only recombining obvious patterns.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making creativity explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for creativity with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'Creativity' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates creativity on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.

- [ ] Repeat key tests across at least three separate runs to check stability.
- [ ] Log every failure with the input, observed output, expected behavior, and probable cause.
- [ ] Confirm no previously approved stage regressed materially after this capability was added.
- [ ] Test interrupted runs, invalid tools, and missing permissions.
- [ ] Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- [ ] You can explain in plain language what counts as success for creativity.
- [ ] The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- [ ] Results are good enough that you would trust the feature in its current bounded scope.
- [ ] The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- [ ] Passes only on cherry-picked prompts or a single benchmark family.
- [ ] Failure cases are not cataloged well enough to reproduce and fix.
- [ ] You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- [ ] The new stage materially worsens any previously approved stage.

Stage 36 — Scientific Reasoning

Stage intent	Form hypotheses, design tests, interpret evidence, and update beliefs.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making scientific reasoning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- [ ] Stage specification for scientific reasoning with a strict in-scope / out-of-scope boundary.
- [ ] A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- [ ] A reproducible test script or benchmark harness dedicated to this stage.
- [ ] A failure log capturing at least the top recurring breakdowns observed during testing.
- [ ] An execution boundary defining what the system may do automatically and what still requires user confirmation.
- [ ] Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- [ ] Define the operational meaning of 'Scientific Reasoning' in one page or less so the stage is testable instead of vague.
- [ ] Set one minimum success metric and one stretch metric before implementation begins.
- [ ] Implement the smallest version that can pass a controlled test without depending on later stages.

- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates scientific reasoning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for scientific reasoning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 37 — Mathematical Competence

Stage intent	Carry out formal symbolic, numerical, and quantitative reasoning.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making mathematical competence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for mathematical competence with a strict in-scope / out-of-scope boundary.

- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'Mathematical Competence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates mathematical competence on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for mathematical competence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 38 — Strategic Reasoning

Stage intent	Plan under competition, cooperation, incentives,

	and adversarial pressure.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making strategic reasoning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for strategic reasoning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** An execution boundary defining what the system may do automatically and what still requires user confirmation.
- **[ ]** Action logs that capture plan, tool calls, and final outcome for review.

Implementation checklist

- **[ ]** Define the operational meaning of 'Strategic Reasoning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Separate planner logic from executor logic so plans can be inspected before or after action.
- **[ ]** Add stop conditions, timeouts, and budget limits before allowing longer runs.
- **[ ]** Design a safe fallback when the plan becomes invalid or incomplete mid-execution.

Test checklist

- **[ ]** Verify the system demonstrates strategic reasoning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test interrupted runs, invalid tools, and missing permissions.
- **[ ]** Verify that the system stops when a guardrail, timeout, or approval rule is triggered.

Approval gate

- **[ ]** You can explain in plain language what counts as success for strategic reasoning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system remains controllable; it does not keep acting when it should stop or ask.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Social alignment, safety, and epistemics

These stages focus on human fit: social modeling, personalization, alignment, safety, ethics, truthfulness, and confidence control.

Gate discipline: This cluster is about trust. A stage is incomplete if the model sounds good but cannot justify or bound what it is saying.

Stage 39 — Social Intelligence

Stage intent	Model social norms, roles, incentives, and interpersonal dynamics.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making social intelligence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for social intelligence with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Social Intelligence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates social intelligence on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for social intelligence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 40 — Theory of Mind

Stage intent	Estimate what different agents know, believe, want, and misunderstand.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making theory of mind explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for theory of mind with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Theory of Mind' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.

- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates theory of mind on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for theory of mind.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 41 — Personalization

Stage intent	Adapt behavior to a specific user's preferences, level, and objectives.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making personalization explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for personalization with a strict in-scope / out-of-scope boundary.

- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Personalization' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates personalization on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for personalization.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 42 — Value Alignment

Stage intent	Act in ways that remain faithful to intended

	human goals and constraints.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making value alignment explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for value alignment with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Value Alignment' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates value alignment on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for value alignment.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 43 — Safety Awareness

Stage intent	Recognize and avoid harmful actions, unsafe advice, and hazardous failure modes.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making safety awareness explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for safety awareness with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Safety Awareness' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates safety awareness on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for safety awareness.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 44 — Ethical Sensitivity

Stage intent	Notice morally relevant tradeoffs and escalate ambiguous cases appropriately.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making ethical sensitivity explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for ethical sensitivity with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Ethical Sensitivity' in one page or less so the stage is testable instead of vague.

- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates ethical sensitivity on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for ethical sensitivity.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 45 — Truth-Seeking Behavior

Stage intent	Prefer accurate, evidence-grounded answers over persuasive but unsupported ones.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making truth-seeking behavior explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for truth-seeking behavior with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Truth-Seeking Behavior' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates truth-seeking behavior on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for truth-seeking behavior.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 46 — Hallucination Resistance

Stage intent	Reduce fabricated facts and unsupported claims under pressure or ambiguity.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making hallucination resistance explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for hallucination resistance with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Hallucination Resistance' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates hallucination resistance on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for hallucination resistance.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 47 — Calibration

Stage intent	Match confidence levels to actual correctness rates.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making calibration explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for calibration with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Calibration' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.

- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates calibration on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for calibration.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 48 — Reliability

Stage intent	Produce stable, repeatable performance across runs and ordinary operating conditions.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making reliability explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for reliability with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** Policy rules describing refusal, escalation, uncertainty, and user-preference handling.
- **[ ]** A red-team set of edge cases where the system must stay helpful without becoming unsafe or deceptive.

Implementation checklist

- **[ ]** Define the operational meaning of 'Reliability' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Encode the user interest, policy boundary, and uncertainty handling in the same decision loop.
- **[ ]** Prefer explicit caveats and escalation over silent guesswork on high-risk cases.
- **[ ]** Test whether the model stays aligned under adversarial prompting, not only cooperative prompts.

Test checklist

- **[ ]** Verify the system demonstrates reliability on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Include adversarial, manipulative, or emotionally loaded prompts in the evaluation set.
- **[ ]** Check whether confidence language matches actual correctness and evidence quality.

Approval gate

- **[ ]** You can explain in plain language what counts as success for reliability.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The system stays helpful without becoming overconfident, manipulative, or unsafe.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Robustness, systems maturity, and scale

These stages harden the system so it remains consistent, reviewable, extensible, and broadly useful without brittle task-by-task engineering.

Gate discipline: These stages turn a lab demo into a durable system. Promotion should require evidence from repeated runs, not intuition.

Stage 49 — Robustness

Stage intent	Remain functional under noise, ambiguity, adversarial inputs, and distribution shift.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making robustness explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for robustness with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Robustness' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates robustness on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.

- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for robustness.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 50 — Generalization to Novelty

Stage intent	Handle problems and contexts that differ materially from training examples.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making generalization to novelty explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for generalization to novelty with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Generalization to Novelty' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.

- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates generalization to novelty on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for generalization to novelty.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 51 — Compositionality

Stage intent	Combine smaller learned skills into larger new behaviors.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making compositionality explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for compositionality with a strict in-scope / out-of-scope boundary.

- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Compositionality' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates compositionality on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for compositionality.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 52 — Temporal Coherence

Stage intent	Stay consistent in facts, goals, and plans over long

	sessions or tasks.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making temporal coherence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for temporal coherence with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Temporal Coherence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates temporal coherence on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for temporal coherence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.

- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 53 — Identity and Role Stability

Stage intent	Maintain the intended role, boundaries, and operating persona over time.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making identity and role stability explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for identity and role stability with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Identity and Role Stability' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates identity and role stability on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for identity and role stability.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 54 — Self-Contained Task Management

Stage intent	Track open loops, priorities, deadlines, and dependencies internally.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making self-contained task management explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for self-contained task management with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.

- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Self-Contained Task Management' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates self-contained task management on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for self-contained task management.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 55 — Explainability

Stage intent	Provide useful explanations for outputs, decisions, and actions.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making

	explainability explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for explainability with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Explainability' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates explainability on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for explainability.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.

- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 56 — Auditability

Stage intent	Record enough evidence that behavior can be reviewed and verified later.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making auditability explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for auditability with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Auditability' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates auditability on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.

- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for auditability.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 57 — Preference Learning

Stage intent	Infer latent user preferences from behavior, corrections, and context.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making preference learning explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for preference learning with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Preference Learning' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.

- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates preference learning on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for preference learning.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 58 — Negotiation of Ambiguity

Stage intent	Spot unclear requests and resolve them safely and efficiently.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making negotiation of ambiguity explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for negotiation of ambiguity with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.

- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Negotiation of Ambiguity' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates negotiation of ambiguity on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for negotiation of ambiguity.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 59 — Embodied Competence

Stage intent	Perceive and act in the physical world if physical-world deployment is in scope.

Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making embodied competence explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for embodied competence with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Embodied Competence' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates embodied competence on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for embodied competence.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 60 — Resource Awareness

Stage intent	Reason about limits in time, compute, memory, money, and energy.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making resource awareness explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for resource awareness with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Resource Awareness' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates resource awareness on at least 10 hand-curated examples.

- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for resource awareness.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 61 — Self-Improvement Ability

Stage intent	Improve methods, prompts, data use, or policies without degrading control.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making self-improvement ability explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for self-improvement ability with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Self-Improvement Ability' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates self-improvement ability on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for self-improvement ability.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 62 — Cross-Domain Integration

Stage intent	Unify language, memory, reasoning, perception, and action into one system.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making cross-domain integration explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or

	reject.

Minimum deliverables

- **[ ]** Stage specification for cross-domain integration with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Cross-Domain Integration' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates cross-domain integration on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for cross-domain integration.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.

- **[ ]** The new stage materially worsens any previously approved stage.

Stage 63 — Human-Comparable Task Range

Stage intent	Demonstrate a task range similar to an educated human generalist.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making human-comparable task range explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for human-comparable task range with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Human-Comparable Task Range' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.
- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates human-comparable task range on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.

- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for human-comparable task range.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Stage 64 — Minimal Dependence on Task-Specific Engineering

Stage intent	Need little bespoke scaffolding when new tasks are introduced.
Why this stage matters	This stage reduces the gap between a narrow assistant and a more general system by making minimal dependence on task-specific engineering explicit, testable, and reviewable.
Evidence to collect	Demo transcript, stage benchmark results, failure log, and a written decision to approve, revise, or reject.

Minimum deliverables

- **[ ]** Stage specification for minimal dependence on task-specific engineering with a strict in-scope / out-of-scope boundary.
- **[ ]** A runnable implementation in the main Python codebase with feature flags or stage toggles disabled for unfinished work.
- **[ ]** A reproducible test script or benchmark harness dedicated to this stage.
- **[ ]** A failure log capturing at least the top recurring breakdowns observed during testing.
- **[ ]** A stress-test bundle covering perturbations, long runs, and regression checks.
- **[ ]** A release gate that blocks promotion when this stage materially degrades earlier approved stages.

Implementation checklist

- **[ ]** Define the operational meaning of 'Minimal Dependence on Task-Specific Engineering' in one page or less so the stage is testable instead of vague.
- **[ ]** Set one minimum success metric and one stretch metric before implementation begins.
- **[ ]** Implement the smallest version that can pass a controlled test without depending on later stages.

- **[ ]** Instrument the feature so you can inspect inputs, outputs, intermediate state, and failure reasons.
- **[ ]** Add explicit stage boundaries so unfinished capabilities do not masquerade as completed behavior.
- **[ ]** Run regression checks against all previously approved stages before marking this one complete.
- **[ ]** Quantify failure under stress instead of relying on anecdotal good runs.
- **[ ]** Keep interfaces generic so the capability survives model swaps and infrastructure changes.

Test checklist

- **[ ]** Verify the system demonstrates minimal dependence on task-specific engineering on at least 10 hand-curated examples.
- **[ ]** Run a negative-control set where the model should admit limitation, ask for clarification, or refuse to guess.
- **[ ]** Repeat key tests across at least three separate runs to check stability.
- **[ ]** Log every failure with the input, observed output, expected behavior, and probable cause.
- **[ ]** Confirm no previously approved stage regressed materially after this capability was added.
- **[ ]** Test long sessions, noisy inputs, and version changes where applicable.
- **[ ]** Measure degradation curves, not just binary pass/fail, so you know where the edge begins.

Approval gate

- **[ ]** You can explain in plain language what counts as success for minimal dependence on task-specific engineering.
- **[ ]** The stage can be demonstrated live with a short scripted test and a few unscripted prompts.
- **[ ]** Results are good enough that you would trust the feature in its current bounded scope.
- **[ ]** The capability survives stress, long runs, and normal infrastructure variation with acceptable degradation.

Do not advance if

- **[ ]** Passes only on cherry-picked prompts or a single benchmark family.
- **[ ]** Failure cases are not cataloged well enough to reproduce and fix.
- **[ ]** You cannot tell whether a good result came from the intended capability versus accidental prompt scaffolding.
- **[ ]** The new stage materially worsens any previously approved stage.

Appendix — Master Approval Tracker

Use this appendix as the sign-off ledger. Mark only after the live demo, the dedicated test set, and the regression pass all succeed.

Stage	Ability	Status	Approved by	Notes / Date
1	Broad Generality			
2	Transfer Learning			
3	Rapid Adaptation			
4	Few-Shot or Zero-Shot Competence			
5	Robust Reasoning			
6	Common-Sense Understanding			
7	Abstract Thinking			
8	Causal Reasoning			
9	Long-Horizon Planning			
10	Goal-Directed Behavior			
11	Subgoal Decomposition			
12	Working Memory			
13	Long-Term Memory			
14	Retrieval and Use of Prior Knowledge			
15	World Modeling			
16	Prediction			
17	Uncertainty Awareness			
18	Metacognition			
19	Error Correction			
20	Self-Monitoring			
21	Learning from Experience			
22	Continual Learning			
23	Sample Efficiency			
24	Multimodal Understanding			

25	Multimodal Generation			
26	Language Competence			
27	Pragmatic Communication			
28	Interactive Dialogue			
29	Tool Use			
30	Environment Interaction			
31	Autonomy			
32	Instruction Following			
33	Constraint Satisfaction			
34	General Problem Solving			
35	Creativity			
36	Scientific Reasoning			
37	Mathematical Competence			
38	Strategic Reasoning			
39	Social Intelligence			
40	Theory of Mind			
41	Personalization			
42	Value Alignment			
43	Safety Awareness			
44	Ethical Sensitivity			
45	Truth-Seeking Behavior			
46	Hallucination Resistance			
47	Calibration			
48	Reliability			
49	Robustness			
50	Generalization to Novelty			
51	Compositionality			
52	Temporal			

	Coherence			
53	Identity and Role Stability			
54	Self-Contained Task Management			
55	Explainability			
56	Auditability			
57	Preference Learning			
58	Negotiation of Ambiguity			
59	Embodied Competence			
60	Resource Awareness			
61	Self-Improvement Ability			
62	Cross-Domain Integration			
63	Human-Comparable Task Range			
64	Minimal Dependence on Task-Specific Engineering			